

LẬP TRÌNH WEB NÂNG CAO

Thái Thanh Tùng
FITHOU

BÀI 1: TỔNG QUAN SỰ PHÁT TRIỂN ASP.NET

- ✓ ASP.NET và công nghệ phát triển Web
- ✓ ASP.NET Core MVC
- ✓ Mô hình và cấu trúc ASP.NET MVC

TỔNG QUAN ASP.NET

1. ASP.NET và những công nghệ mới

- ASP.NET là một mô hình chuẩn hỗ trợ phát triển các ứng dụng Web trên nền .NET bao gồm cả các dịch vụ cho phép xây dựng các ứng dụng lớn
- ASP.NET cung cấp 3 frameworks để phát triển ứng dụng Web
 - Web Forms
 - MVC
 - Web Pages
- Cả 3 frameworks đều đã ổn định và được cập nhật hỗ trợ trong các phiên bản mới

Các frameworks hướng tới các loại ứng dụng khác nhau, tuy nhiên việc phát triển phụ thuộc vào kinh nghiệm và sự quen thuộc.

TỔNG QUAN ASP.NET

1.1. ASP.NET Web Forms (.aspx)

- Công nghệ này hướng tới các developers quen với lập trình điều khiển tương tự như trên Window Application, mô hình lập trình hướng giao diện (kéo và thả), và thường được gọi là công nghệ phát triển ứng dụng nhanh (RAD: Rapid Application Development). Các đặc trưng:
 - Mô hình sự kiện
 - Các điều khiển Server
 - Hỗ trợ các điều khiển dữ liệu
 - Quản lý trạng thái giao thức HTTP
- Mô hình này phù hợp với các nhóm phát triển và thiết kế khi cần phải phát triển những ứng dụng lớn bao gồm nhiều components sẽ yêu cầu ít thời gian **code** và **design**

TỔNG QUAN ASP.NET

1.2. ASP.NET MVC (.cshtml, .vbhtml)

- Công nghệ này hướng tới các developers ưa thích lập trình theo mô hình và nguyên lý, mô hình lập trình theo hướng có kiểm soát (bởi các test case), tách biệt chức năng và thường tách biệt hoàn toàn lớp giao diện (presentation) và lớp chức năng (business logic).
- Mô hình này phù hợp với nhiều nhóm phát triển và thiết kế tách biệt vì các lớp Model, View, Controller được phân chia tách biệt trong ứng dụng. Ứng dụng lớn có thể tách biệt nhóm thực hiện các unit độc lập
- Mô hình MVC cho phép làm việc trực tiếp với HTML và HTTP, trong phiên bản MVC 4 trở lên còn cho phép phát triển các dịch vụ trên HTTP hỗ trợ nhiều nền tảng Client

TỔNG QUAN ASP.NET

1.3. ASP.NET Web Pages (.cshtml, .vbhtml)

- Công nghệ này hướng tới ứng dụng nhỏ, đơn giản tương tự như PHP. Đây là mô hình đơn giản nhất và dễ thích ứng cho những ai mới làm quen với ASP.NET. Mô hình này cho phép phát triển ứng dụng nhanh (RAD) nhưng đơn giản hơn rất nhiều so với mô hình Web Form
- Mô hình này phù hợp với việc phát triển các ứng dụng nhỏ không yêu cầu quá nhiều chức năng, người dùng chỉ tạo trang HTML và thêm các điều khiển helper để thực hiện chức năng đơn giản như login facebook, hiển thị bản đồ, và hiển thị dữ liệu...

TỔNG QUAN ASP.NET

1.4. ASP.NET và Mã nguồn mở

Từ tháng 3, 2012, Microsoft đã đăng ký bản quyền open source cho ASP.NET MVC 4, ASP.NET Web API, and ASP.NET Web Pages v2 (Razor syntax) theo đăng ký [Apache 2.0 license](#). ASP.NET Web Forms chưa được đăng ký bản quyền

Tham khảo:

1. [ASP.NET MVC, Web API, Razor and Open Source](#) (ScottGu)
2. [ASP.NET MVC 5, ASP.NET Web API and ASP.NET Web Pages v2 \(Razor\)](#) (Scott Hanselman)

TỔNG QUAN ASP.NET

2. ASP.NET

2.1. Web Application và Web Site Projects

Trong VS.NET ta có thể lựa chọn tạo Web Application hoặc Web Site. Mỗi loại sẽ có đặc trưng riêng và nên lựa chọn phù hợp vì việc chuyển đổi giữa hai loại sẽ phức tạp. Người mới phát triển nên lựa chọn Web Application vì các tính năng mới thường phát triển cho Application. Khi cần lựa chọn ta xem xét các yếu tố sau

Web Application	Web Site Project
- Thực hiện unit test trong tệp mã nguồn của lớp dữ liệu - Tạo một assembly và xác định tên, phiên bản cho toàn bộ ứng dụng - Khi không muốn upload file nguồn lên server ...	- Trong mã nguồn gồm cả C# và VB.Net - Có khả năng mở ứng dụng và update các file ASPX trên server và update real time bằng cách sử dụng dịch vụ FTP trong VS.NET - Lưu mã nguồn trên server để backup

TỔNG QUAN ASP.NET

Khác biệt	Web application projects	Web site projects
Cấu trúc tệp Project	Tệp project (.csproj or .vbproj) lưu thông tin của project, như danh sách tệp, tham chiếu giữa các projects.	Không có tệp project. Tất cả các tệp trong cấu trúc thư mục được bao gồm trong Web site.
Biên dịch	• Ta phải biên dịch các file mã nguồn tại máy nguồn phát triển ứng dụng • Các file nguồn được biên dịch thành 1 assembly (ngoại trừ tệp .aspx and .ascx files)	• Mã nguồn được biên dịch động (tự động) bởi ASP.NET trên server khi nhận request đầu tiên sau khi cài đặt hoặc cập nhật. • Ta cũng có thể biên dịch Website trên máy nguồn phát triển. • Mặc định các file nguồn được biên dịch thành nhiều assembly
Namespaces	Namespaces mặc định được thêm vào các trang, điều khiển, và lớp dữ liệu một cách rõ ràng	Namespaces không được thêm vào các trang, điều khiển, và lớp dữ liệu, người lập trình phải nhập thủ công.
Triển khai	• Sau khi biên dịch ứng dụng thành assembly, ta có thể copy assembly lên máy chủ.. • Visual Studio cung cấp công cụ tích hợp trong Web Deploy (IIS web deployment tool) tự động triển khai ứng dụng.	• Ta sao chép toàn bộ tệp nguồn của ứng dụng lên máy được cài đặt IIS. • Nếu ta biên dịch trước tại máy nguồn phát triển thì sao chép tất cả các assembly. • Visual Studio cung cấp công cụ tích hợp trong Web Deploy (IIS web deployment tool) tự động triển khai ứng dụng.

ASP.NET core MVC

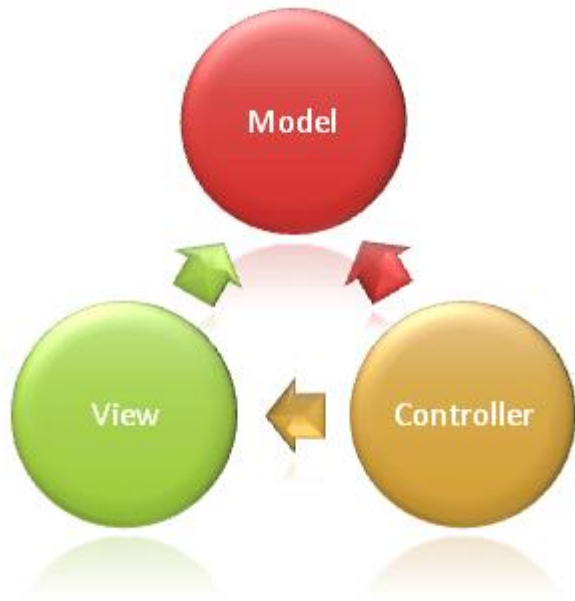
2.2. ASP.NET core MVC

MVC

- MVC là một mẫu (mô hình) kiến trúc thiết kế phần mềm được sử dụng để phát triển các ứng dụng tương tác có liên quan đến tương tác người dùng và xử lý sự kiện dựa trên tương tác người dùng.
- Sử dụng cho các loại ứng dụng web-based, application-based hoặc thiết bị di động có liên quan đến tương tác của users.
- Mẫu thiết kế MVC được giới thiệu vào những năm 1970s, về cơ bản chia ứng dụng thành 3 thành phần chính là Model, View và Controller.
- Mục tiêu chính của mẫu thiết kế MVC là tách biệt các thành phần ứng dụng (giao diện, nghiệp vụ, và xử lý dữ liệu). Nhờ đó, việc bảo trì và test ứng dụng trở nên đơn giản và dễ dàng

ASP.NET core MVC

Mô hình kiến trúc MVC



Hình 1: Mô hình kiến trúc MVC

Model:

- Model trong ứng dụng MVC là thành phần nghiệp vụ business logic. Các chức năng nghiệp vụ hoặc thao tác với dữ liệu được triển khai trong Model.

View:

- Views chịu trách nhiệm hiển thị dữ liệu giao diện người dùng của thành phần Presentation. Thường sử dụng [Razor view engine](#) để nhúng mã .NET trong trang HTML. Mã chương trình chủ yếu để hiển thị dữ liệu, hạn chế tối thiểu các nghiệp vụ business logic trong View. Trong trường hợp phải xử lý nghiệp vụ logic trong file view thì nên sử dụng [View Component](#), ViewModel.

Controller:

- Controllers là thành phần xử lý tương tác với user, làm việc với model, và lựa chọn view để render dữ liệu hiển thị.

Trong mô hình MVC: Controller là điểm kích hoạt ứng dụng, chịu trách nhiệm gọi các Model xử lý và chọn View để render dữ liệu

ASP.NET core MVC

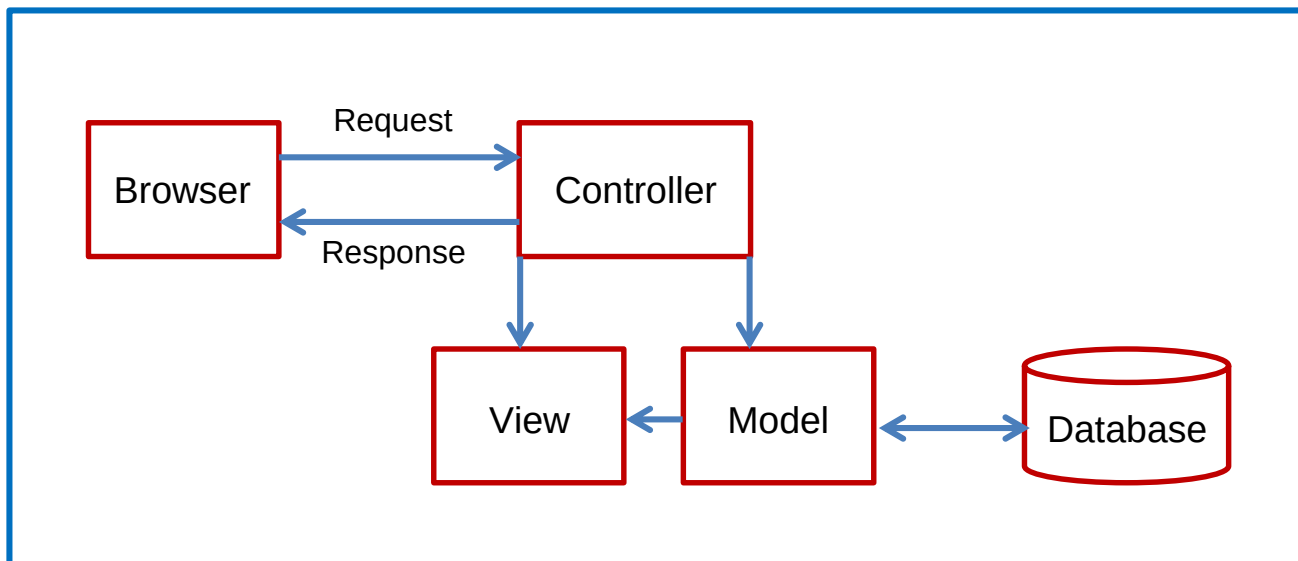
2.2. ASP.NET core MVC

ASP.NET core MVC

- Framework dựa trên mô hình thiết kế **MVC Pattern** (Model-View-Controller). ASP.NET MVC là một Framework trong khi MVC là một Design Pattern.
- ASP.NET MVC Framework là một cách tiếp cận thay thế cho ASP.NET Web Forms Framework truyền thống. Được xây dựng trên .NET Framework, và thừa hưởng hầu hết tất cả các tính năng của ASP.NET khi làm việc với ứng dụng MVC.
- ASP.NET Core MVC framework là nền tảng nhẹ, nguồn mở, cho phép test giao diện nhanh tối ưu hóa sử dụng với ASP.NET Core. Ứng dụng được phát triển trực quan và tách biệt các modules ứng dụng.

ASP.NET core MVC

Mô hình kiến trúc ASP.NET core MVC:



Hình 1: Mô hình kiến trúc ASP.NET core MVC
Client và Server giao tiếp thông qua HTTPRequest - HTTPResponse

ASP.NET core MVC

Mô hình kiến trúc ASP.NET core MVC:

Controller là bộ điều khiển là thành phần trong ứng dụng ASP.NET MVC thực sự nhận yêu cầu HTTPRequest đến và sau đó xử lý yêu cầu đó. Để xử lý yêu cầu HTTPRequest, các hoạt động sau sẽ được thực hiện:

1. Controller tạo đối tượng Model được yêu cầu bởi một View. Model là thành phần trong MVC chứa một tập hợp các lớp để biểu diễn dữ liệu hoặc dữ liệu nghiệp vụ cũng như logic để xử lý dữ liệu.
2. Sau đó, bộ điều khiển chọn một View để hiển thị dữ liệu hoặc dữ liệu nghiệp vụ. Chú ý: khi chọn chế độ View, trách nhiệm của Controller là truyền dữ liệu từ Model.
3. Trong mô hình MVC, nhiệm vụ cơ bản nhất của một View là hiển thị dữ liệu Model. Vì vậy, View sẽ tạo tệp HTML cần thiết sẽ hiển thị dữ liệu Model hoặc dữ liệu nghiệp vụ. Khi HTML được tạo bởi View, thì HTML đó sẽ được gửi đến máy khách thông qua HTTPResponse

ASP.NET core MVC

2.3. Cài đặt và Thiết lập ASP.NET core MVC

Cài đặt ASP.NET MVC

- Download cài đặt .Net SDK (6.0 long term hoặc 7.0 standard support)
- Visual Studio Code
- Download cài đặt [Visual Studio 2022](#) (hoặc trước đó) với bộ ASP.NET và Web Development

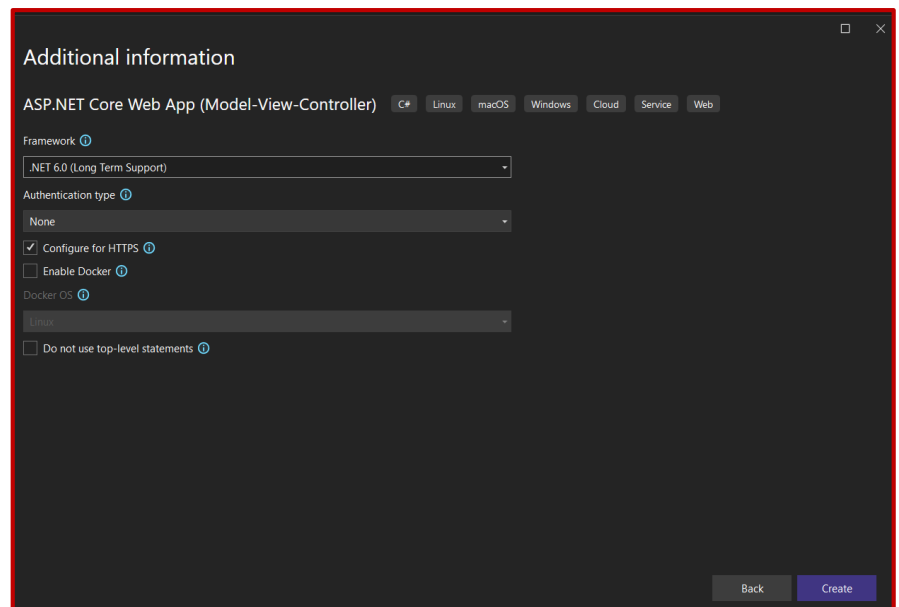
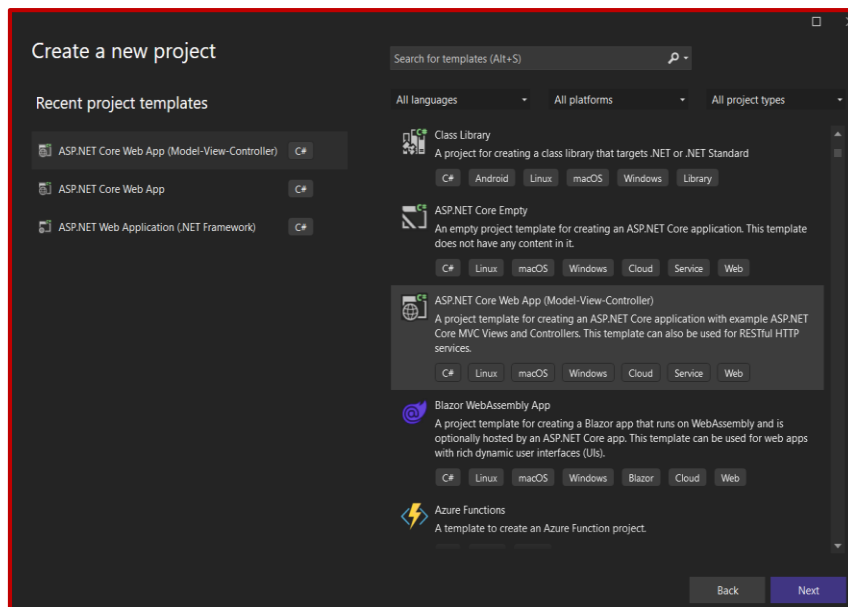
Thiết lập ASP.NET core MVC Application

- Khởi động Visual Studio và tạo Project mới: [Create a new project](#)
- Chọn Web trong phần Templates
- Chọn kiểu Project
- Chọn .NET Framework (6.0 hoặc 7.0)
- Đặt tên project xác định tạo Solution cho project
- Click OK to create the project

Chú ý: Tên ứng dụng phân biệt chữ hoa chữ thường để tham chiếu với tên `namespace` khi sao chép, chuyển mã nguồn

ASP.NET core MVC

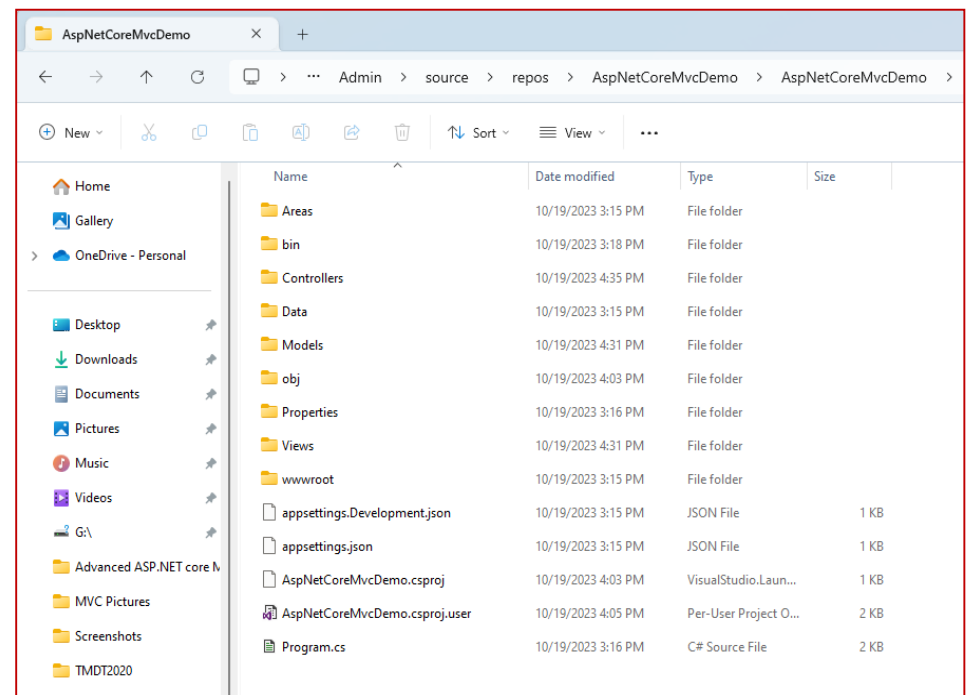
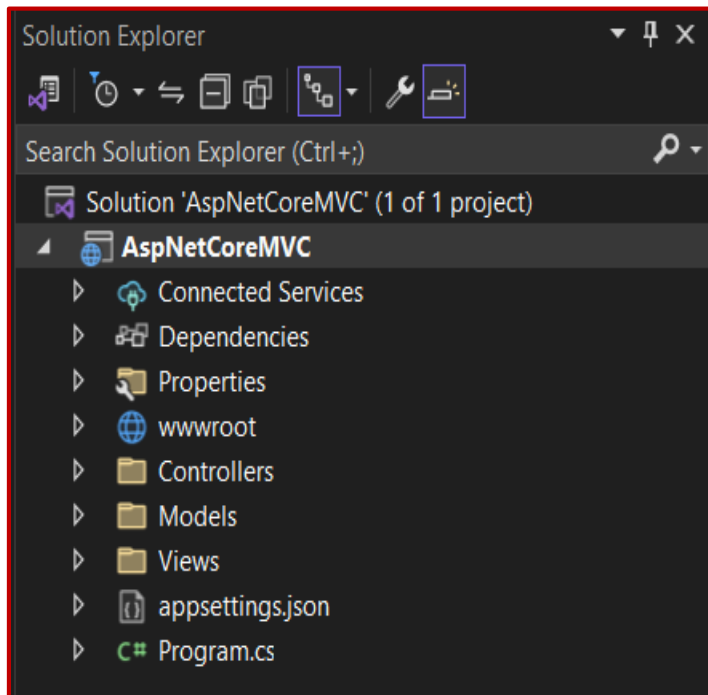
2.3. Cài đặt và Thiết lập ASP.NET core MVC



Config Project: Chọn **No Authentication** vì chưa sử dụng an ninh xác thực định danh

ASP.NET core MVC

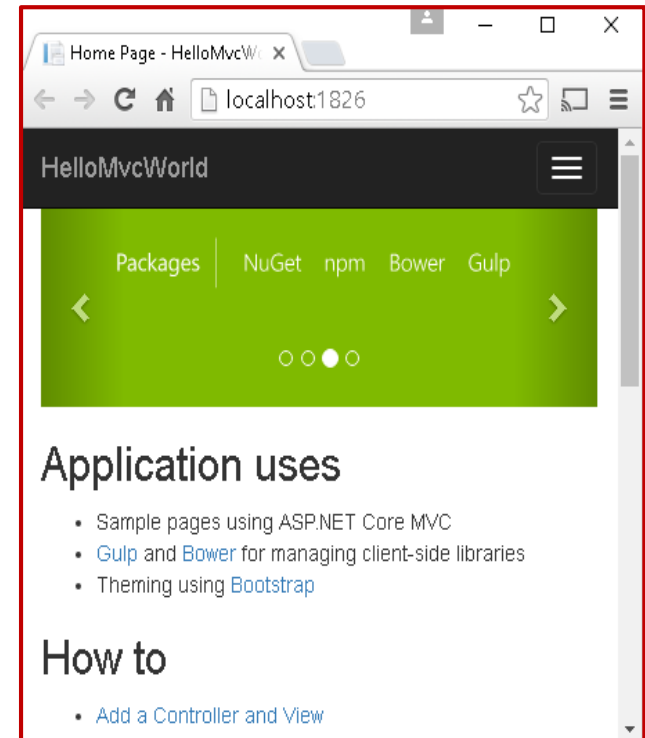
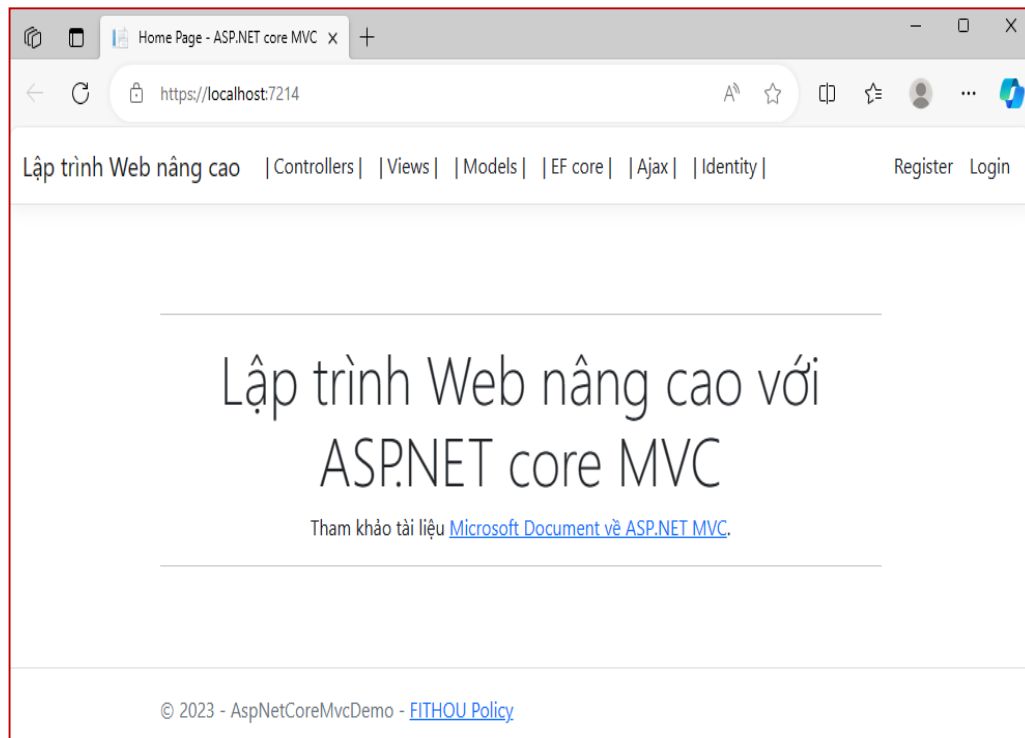
2.3. Cài đặt và Thiết lập ASP.NET core MVC



Cấu trúc Logic và cấu trúc thư mục Vật lý của Project

ASP.NET core MVC

2.3. Cài đặt và Thiết lập ASP.NET core MVC



MVC 5 bao gồm JS, CSS cho Bootstrap để tạo trang web Responsive hiển thị theo trình duyệt của thiết bị (Mobile version)

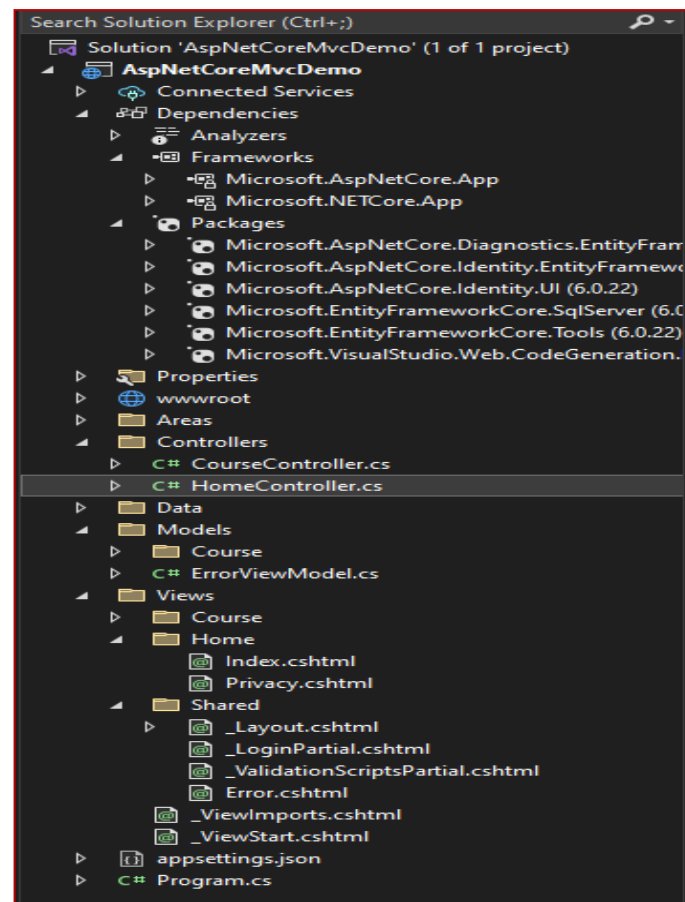
Cấu trúc ASP.NET core MVC

3. Cấu trúc ứng dụng ASP.NET core MVC

Controllers Folder chứa các tệp điều khiển (*.cs) của ứng dụng. Controllers là những lớp dữ liệu kế thừa từ lớp cơ sở Controller class. Lớp dữ liệu này sẽ nhận và xử lý yêu cầu từ phía người dùng (HTTPRequests và trả lại HTTPResponse). Tên điều khiển sẽ kèm theo “*Controller*”.

Models Folder chứa tất cả các file lớp dữ liệu (*.cs) và xử lý dữ liệu (data, data access layer, business logic layer).

Views Folder chứa các tệp giao diện UI (*.cshtml) HTML cùng mã ngôn ngữ (C#). Thư mục Views sẽ tạo các thư mục con cho mỗi Controller. Ví dụ, các tệp .cshtml files trong HomeController sẽ đặt trong View => Home. Ta có thư mục **Shared** chứa các Views sử dụng chung như Error, Layout, Viewports ...

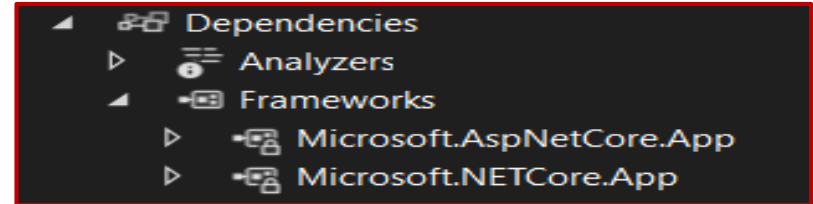


Cấu trúc logic trên IIS (Visual Studio)

Cấu trúc ASP.NET core MVC

3.1. Cấu hình ứng dụng

- **Dependencies:** chứa các dịch vụ Middleware của Microsoft hoặc bên thứ 3 được đăng ký sử dụng trong project
- **program.cs** là tệp cấu hình rút gọn của .Net core, khai báo các Services Middleware của Microsoft và bên thứ 3.
- **appsettings.json** tệp dữ liệu định dạng JSON, để lưu trữ dữ liệu configuration, biến môi trường, và User Secrets



Dependency Injection Services

```
1 using AspNetCoreMvcDemo;
2 using Microsoft.AspNetCore.Identity;
3 using Microsoft.EntityFrameworkCore;
4
5 var builder = WebApplication.CreateBuilder(args);
6
7 // Add services to the container.
8 var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
9 builder.Services.AddDbContext<ApplicationDbContext>(options =>
10     options.UseSqlServer(connectionString));
11 builder.Services.AddDatabaseDeveloperPageExceptionFilter();
12
13 builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true)
14     .AddEntityFrameworkStores<ApplicationDbContext>();
15 builder.Services.AddControllersWithViews();
16
17 var app = builder.Build();
18
19 // Configure the HTTP request pipeline.
20 if (app.Environment.IsDevelopment())
21 {
22     app.UseMigrationsEndPoint();
23 }
24 else
25 {
26     app.UseExceptionHandler("/Home/Error");
27     // The default HSTS value is 30 days. You may want to change this for production scenarios, see https://aka.ms/aspnetcore-hsts
28     app.UseHsts();
29 }
30
31 app.UseHttpsRedirection();
32 app.UseStaticFiles();
33
34 app.UseRouting();
35
36 app.UseAuthentication();
37 app.UseAuthorization();
38
39 app.MapControllerRoute(
40     name: "default",
41     pattern: "[controller=Home]/[action=Index]/[id?]*");
42 app.MapRazorPages();
43
44 app.Run();
45
```

program.cs cấu hình services và extension

Cấu trúc ASP.NET core MVC

Middleware Components

Middleware components là các module chương trình chịu trách nhiệm xử lý các tác vụ cụ thể trong quá trình xử lý các yêu cầu HTTP đến và đi. Chúng cho phép ta thiết lập các chức năng khác nhau vào ứng dụng của mình, chẳng hạn như xác thực, định tuyến, ghi nhật ký. Một số services thường dùng:

- **UseAuthentication:** Xác thực định danh người dùng
- **UseHttpsRedirection:** Chuyển HTTPRequests sang chế độ HTTPS
- **UseHttpLogging:** Ghi vào log các HTTP Requests, Responses
- **UseDeveloperExceptionPage:** Gọi trang thông báo lỗi tập trung của ứng dụng
- **UseExceptionHandler:** Bắt lỗi exceptions, ghi log, và thực thi request với pipeline khác
- **UseStaticFiles:** Xử lý static files như Images, Javascript, hoặc CSS, etc
- **UseAuthorization:** Kiểm tra quyền truy cập người dùng tới dữ liệu

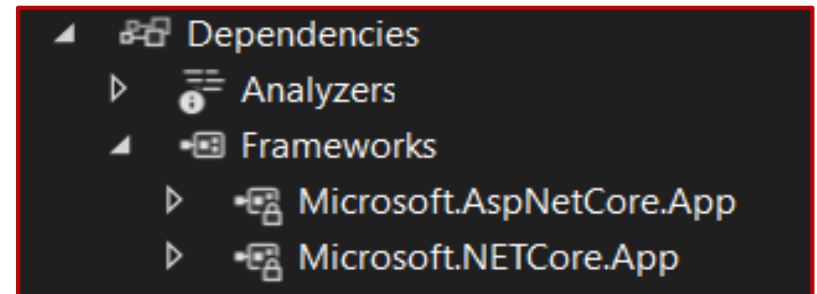
UseRouting: Thiết lập định tuyến End Points Routing Middleware cho Request Pipeline. Nếu định nghĩa bộ định tuyến sử dụng Pattern hoặc thuộc tính routing thì bắt buộc phải sử dụng UseRouting Middleware Component.

Cấu trúc ASP.NET core MVC

Dependencies:

Các thành phần chính trong Dependencies

- **NuGet Packages:** Chứa các gói thư viện từ NuGet. Một số gói phổ biến:
 - **Microsoft.EntityFrameworkCore:** Thư viện ORM để làm việc với cơ sở dữ liệu.
 - **Microsoft.EntityFrameworkCore.Sql Server:** Provider hỗ trợ SQL Server.
 - **Microsoft.AspNetCore.Mvc:** Framework MVC.
 - **Swashbuckle.AspNetCore:** Thư viện cho Swagger API documentation.
- **SDK:** Thư viện và framework đi kèm với .NET SDK, cung cấp công cụ phát triển.
- **Project Dependencies:** Các dự án khác mà dự án hiện tại tham chiếu tới.



Dependency quản lý các gói thư viện
NuGet Package Manager

Cấu trúc ASP.NET core MVC

Properties: chứa tệp cấu hình

launchSettings.json: cấu hình cách ứng dụng được khởi chạy trong môi trường phát triển (development environment)

Thành phần chính gồm:

- **commandName:** Xác định IISExpress hay Project
- **applicationUrl:** địa chỉ Url xử lý request
- **environmentVariables:** biến môi trường (ASPNETCORE_ENVIRONMENT)

```
json
{
  "profiles": {
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "MyApp": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "applicationUrl": "https://localhost:5001;http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Cấu trúc ASP.NET core MVC

Connected Services: thêm dịch vụ hoặc API từ bên thứ ba

Dịch vụ phổ biến:

- **MS Azure:** AppServices, SQL-DB
- **WCF:** Kết nối dịch vụ WCF
- **REST API:** Tích hợp API bên thứ ba

Ví dụ: Kết nối DB

Trong tệp appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=tcp:yourserver.database.
        windows.net,1433;Database=yourdb;
        User Id=youruser;Password=yourpassword;"
  }
}
```

Cập nhật DbContext

```
services.AddDbContext<ApplicationDbContext>
(options => options.UseSqlServer(Configuration.
  GetConnectionString("DefaultConnection")));
```

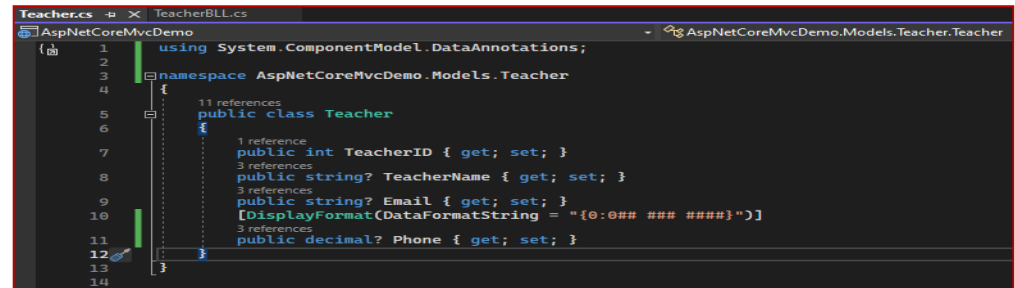

ASP.NET core MVC Application

3.2. Controllers, Models, Views

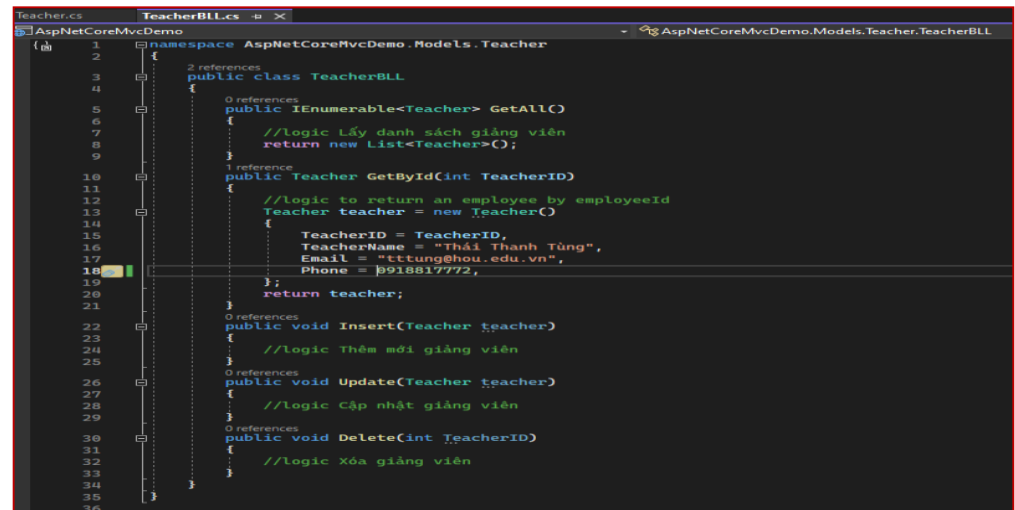
Ví dụ: mô hình MVC

Models:

- Lớp dữ liệu: **Teacher**
- Lớp xử lý dữ liệu:
TeacherBLL



```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace AspNetCoreMvcDemo.Models.Teacher
4 {
5     public class Teacher
6     {
7         public int TeacherID { get; set; }
8         public string? TeacherName { get; set; }
9         public string? Email { get; set; }
10        [DisplayFormat(DataFormatString = "{0:0## ### ####}")]
11        public decimal? Phone { get; set; }
12    }
13 }
14
```



```
1 namespace AspNetCoreMvcDemo.Models.Teacher
2 {
3     public class TeacherBLL
4     {
5         public IEnumerable<Teacher> GetAll()
6         {
7             //logic Lấy danh sách giảng viên
8             return new List<Teacher>();
9         }
10        public Teacher GetById(int TeacherID)
11        {
12            //logic to return an employee by employeeId
13            Teacher teacher = new Teacher()
14            {
15                TeacherID = TeacherID,
16                TeacherName = "Thái Thanh Tùng",
17                Email = "tthung@hou.edu.vn",
18                Phone = 8918817772,
19            };
20            return teacher;
21        }
22        public void Insert(Teacher teacher)
23        {
24            //logic Thêm mới giảng viên
25        }
26        public void Update(Teacher teacher)
27        {
28            //logic Cập nhật giảng viên
29        }
30        public void Delete(int TeacherID)
31        {
32            //logic Xóa giảng viên
33        }
34    }
35 }
36
```

ASP.NET core MVC Application

Views:

- Tập **Index.cshtml** hiển thị dữ liệu
- Theo **tiêu chuẩn Razor** để nhúng mã chương trình vào mã HTML

```
TeacherDetails.cshtml X
1 @model.AspNetCoreMvcDemo.Models.Teacher.Teacher
2
3 @{
4     ViewData["Title"] = "Teacher Details";
5 }
6
7 <h1>Thông tin giảng viên</h1>
8 <hr />
9 <div>
10     <dl class="row">
11         <dt class="col-sm-2">
12             @Html.DisplayNameFor(model => model.TeacherName)
13         </dt>
14         <dd class="col-sm-10">
15             @Html.DisplayFor(model => model.TeacherName)
16         </dd>
17         <dt class="col-sm-2">
18             @Html.DisplayNameFor(model => model.Email)
19         </dt>
20         <dd class="col-sm-10">
21             @Html.DisplayFor(model => model.Email)
22         </dd>
23         <dt class="col-sm-2">
24             @Html.DisplayNameFor(model => model.Phone)
25         </dt>
26         <dd class="col-sm-10">
27             @Html.DisplayFor(model => model.Phone)
28         </dd>
29     </dl>
30 </div>
31 <div>
32     <a asp-action="Index">Quay lại</a>
33 </div>
34
```

ASP.NET core MVC Application

Controllers:

- Lớp điều khiển **TeacherController**

Chú ý: Kế thừa từ lớp cơ sở: `Microsoft.AspNetCore.Mvc.Controller`

```
HomeController.cs
AspNetCoreMvcDemo
using AspNetCoreMvcDemo.Models;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;

namespace AspNetCoreMvcDemo.Controllers
{
    3 references
    public class HomeController : Controller
    {
        8
        private readonly ILogger<HomeController> _logger;
        9
        0 references
        public HomeController(ILogger<HomeController> logger)
        {
            11
            _logger = logger;
            12
        }
        13
        0 references
        public IActionResult Index()
        {
            14
            return View();
            15
        }
        16
        0 references
        public IActionResult Privacy()
        {
            17
            return View();
            18
        }
        19
        0 references
        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            20
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
            21
        }
    }
    22
    23
    24
    25
    26
    27
    28
    29
    30
    31
    32
}
```

```
CourseController.cs
AspNetCoreMvcDemo
using Microsoft.AspNetCore.Mvc;
using AspNetCoreMvcDemo.Models.Teacher;

namespace AspNetCoreMvcDemo.Controllers
{
    0 references
    public class CourseController : Controller
    {
        7
        {
            0 references
            public IActionResult Index()
            {
                8
                return View();
                9
            }
            10
            0 references
            public IActionResult TeacherDetails(int TeacherId)
            {
                11
                TeacherBLL teacherBLL = new TeacherBLL();
                12
                Teacher teacherDetail = teacherBLL.GetById(TeacherId);
                13
                return View(teacherDetail);
                14
            }
            15
        }
        16
        17
        18
        19
        20
    }
}
```

ASP.NET core MVC Application

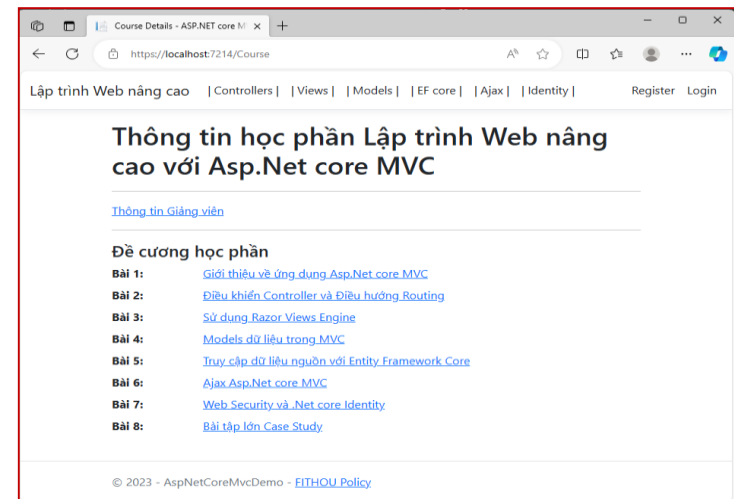
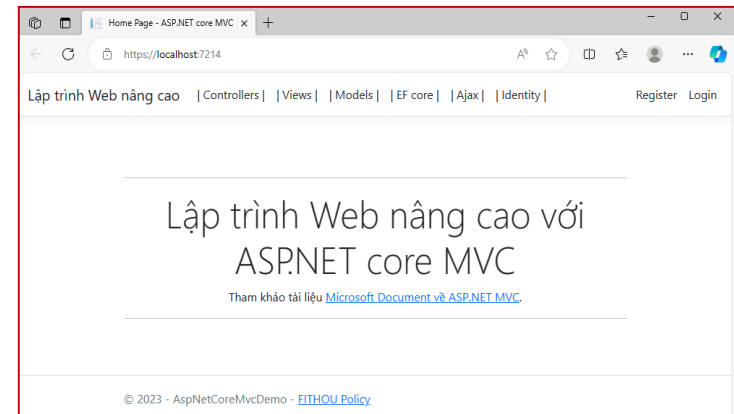
3.3. Thực thi ASP.NET MVC

Mô hình ASP.NET MVC thực thi thông qua Controller xử lý yêu cầu HTTP. Mỗi ứng dụng khởi tạo đều có Điều khiển mặc định **HomeController.cs** và tập View mặc định là **Index.cshtml**. Ta có thể truy cập thông qua IIS từ điều khiển HomeController

Ví dụ: <https://localhost:1234>

- Mỗi Controller có thể được điều hướng truy cập thông qua tên Điều khiển (Controller)
<https://localhost:1234/Course>
- Mỗi phương thức public trong Controller đều có thể được truy cập là một HTTP request

<https://localhost:1234/Course/Details>



KẾT THÚC

Q & A

XIN CHÀO VÀ HẸN GẶP LẠI TUẦN SAU